

Marcar como hecha

11.3. Recuperación de datos enviados en una petición

Ejemplo: Cambiar dirección

Como el siguiente ejemplo de servlet, mostraré una aplicación web donde el usuario debe rellenar un formulario que puede ser relativo a un mensaje en movimiento. La información introducida se envía al servidor donde se validan y si pueden ser aceptados, aparecerá un recibo de que el mensaje que ha sido recibido. En caso contrario, se pedirá al usuario que vuelva a introducir el mensaje. Al abrir la aplicación en el navegador deberá salir un formulario que pida los campos enunciados con un cuadro de texto, y dos botones "reset" y "submit":

Cambiar la dirección

- nombre
- apellido
- dirección
- código y ciudad
- mail
- fecha de cambio de dirección

Si hace clic en el botón Enviar, los datos del formulario se envían al servidor, que podría, por ejemplo, responder con el siguiente mensaje:

Debe incluir la dirección

Intentar otra vez

Puesto que el usuario (en este caso, imaginamos) no ha introducido la dirección completa. A continuación, tendrá que hacer clic en el enlace, y podrá rellenar de nuevo el formulario

Si rellena todos los campos, excepto el de mail que no será obligatorio, y en los que la fecha se introduce en el formato DD-MM-AAAA (es decir, guion entre los campos) y, a continuación, hace clic en Enviar, recibirá un recibo como respuesta (véase más abajo). Se trata, por supuesto, de una aplicación muy sencilla, pero que envía datos del cliente al servidor donde se procesan, tras lo cual el cliente obtiene una respuesta y, en muchos sentidos, es el principio de muchas aplicaciones web.

Gracias por su solicitud

Ha quedado registrada

Nombre Apellido

Dirección

código Ciudad

Nueva dirección registrada en fecha DD-MM-AAAA

La base es un proyecto NetBean, llamado CambiaDireccion, que como el primer proyecto de este capítulo es un proyecto de Aplicación Web. NetBeans crea una página de inicio, llamada index.html, que he modificado a lo siguiente:

```
<!DOCTYPE html>

<html>
<head>
<title>TODO supply a title</title>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
<h1>Cambia Dirección</h1>
<form method="post" action="AddressServlet">
<table>
<tr>
<td>
<label>Nombre: </label></td>
<td><input type="text" id="fname" name="fname"/> </td> </tr> <tr>
<td><label>Apellido: </label></td>
<td><input type="text" id="lname" name="lname"/></td> </tr> <tr>
<td><label>Dirección: </label></td>
<td><input type="text" id="addr" name="addr"/></td> </tr> <tr>
<td><label>Código postal y ciudad: </label></td>
<td><input type="text" id="post" name="post"/></td> </tr><tr>
<td><label>Mail: </label></td>
<td><input type="text" id="mail" name="mail"/></td> </tr> <tr>
<td><label>Fecha de nueva dirección: </label></td>
<td><input type="text" id="date" name="date" /></td> </tr> <tr>
<td><input type="reset" value="Reset"/>
<input type="submit" value="Submit" /></td>
</tr>
</table>
</form>
</body>
</html>
```

Es esta página la que muestra el formulario. Un formulario se define en HTML con un elemento `<form>`. En este caso, los campos del formulario se colocan en una tabla, y está solo para tener los campos bien colocados uno debajo del otro en dos columnas. La tabla tiene 7 filas, y las 6 primeras tienen una etiqueta y un elemento de entrada. A etiqueta muestra un texto, mientras que un elemento de entrada que en este caso tiene el tipo de texto es un campo de entrada. Cada campo de entrada tiene un nombre y, por tanto, un identificador

definido por un atributo `id` y `name`, respectivamente. En este caso, sólo el último es necesario, pero es ligeramente diferente cuando un elemento se identifica por un `id` o un nombre, pero cuando los valores de los elementos se envían al servidor como parte de un formulario, es el atributo `name` el que se utiliza. Sin embargo, si se hace referencia a un elemento desde una hoja de estilo, es el atributo `id` el que se utiliza. Por lo tanto puede ser una buena idea indicar ambos. La última fila de la tabla también tiene dos componentes de entrada, pero son de distinto tipo. Ambos componentes son representados por el navegador como un botón, y el atributo `value` especifica el texto que muestra el botón. El primer componente borra los campos de entrada del formulario, mientras que el otro (componente `submit`) envía el formulario al servidor.

Si consideramos el elemento formulario, tiene dos atributos. `action` indica a qué página del servidor debe enviarse la petición. En este caso, es `AddressServlet`, que es un servlet. El otro método indica qué tipo de petición es y aquí es `POST`. Esto significa que los elementos del formulario se envían junto con la petición como par clave/valor, donde clave es el valor del atributo `name` mientras que valor es el valor del atributo `value`, que para un campo de entrada es el texto introducido.

Como se ha mencionado, `AddressServlet` se refiere a un servlet. Se crea de la misma forma que en el ejemplo anterior, y lo he llamado `AddressServlet`, y en la pantalla `Configure Servlet Deployment`, no he cambiado los valores de `Servlet Name` y `URL Pattern(s)` esta vez, pero he creado un paquete para el servlet. El código completo se muestra a continuación, donde he borrado todos los comentarios así como el último método `getServletInfo()`:

```

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/JSP_Servlet/Servlet.java to edit this template
 */
package changeadress;

import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.util.*;
import java.text.*;
import java.util.regex.*;

/**
 *
 * @author enrique
 */
@WebServlet(name = "AddressServlet", urlPatterns = {"/AddressServlet"})
public class AddressServlet extends HttpServlet {

    /**
     * Processes requests for both HTTP GET and POST
     * methods.
     *
     * @param request servlet request
     * @param response servlet response
     * @throws ServletException if a servlet-specific error occurs
     * @throws IOException if an I/O error occurs
     */
    protected void processRequest(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        try (PrintWriter out = response.getWriter()) {
            /* TODO output your page here. You may use following sample code. */
            out.println("<!DOCTYPE html>");
            out.println("<html>");
            out.println("<head>");
            out.println("<title>AddressServlet</title>");
            out.println("</head>");
            out.println("<body>");
            String fname = request.getParameter("fname");
            String ename = request.getParameter("lname");
            String addr = request.getParameter("addr");
            String post = request.getParameter("post");
            String mail = request.getParameter("mail");
            String date = request.getParameter("date");
            if (fname.length() == 0 || ename.length() == 0) {
                out.println("<h3>Debe ingresar su nombre</h3><br>");
                out.println("<a href='\"index.html\"'>Inténtelo otra vez...</a>");
            } else if (addr.length() == 0 || post.length() == 0) {
                out.println("<h3>Debe ingresar dirección</h3><br>");
                out.println("<a href='\"index.html\"'>Inténtelo otra vez...</a>");
            } else if (mail.length() > 0 && !isMail(mail)) {
                out.println("<h3>Debe ingresar mail adecuado</h3><br>");
                out.println("<a href='\"index.html\"'>Inténtelo otra vez...</a>");
            } else {
                try {
                    SimpleDateFormat formatter = new SimpleDateFormat("dd-mm-yyyy");
                    Date dato = formatter.parse(date);
                    out.println("<h1>Gracias por su inscripción</h1>");
                    out.println("<h3>Se ha registrado</h3>");
                    out.println(fname + " " + ename + "<br>");
                    out.println(addr + "<br>");
                    out.println(post + "<br>");
                }
            }
        }
    }
}

```

?

```

        out.println(mail + "<br>");
        out.println("Nueva dirección validada en fecha: " + date);
    } catch (Exception ex) {
        out.println("<h3>Debe entrar la fecha en formato DD-MM-YYYY</h3><br>");
        out.println("<a href='\"index.html\"'>Inténtelo otra vez..</a>");
    }
}
out.println("</body>");
out.println("</html>");
}
}

private boolean isMail(String mail){
    Pattern pattern = Pattern.compile("^[_A-Za-z0-9-\\+](\\.[_A-Za-z0-9-])*@"
        + "[A-Za-z0-9-](\\.[A-Za-z0-9-])*((\\.[A-Za-z]{2,})$)");
    Matcher mather=pattern.matcher(mail);
    return mather.find();
}

// <editor-fold defaultstate="collapsed" desc="HttpServlet methods. Click on the + sign on the left to edit the code.">
/**
 * Handles the HTTP <code>GET</code> method.
 *
 * @param request servlet request
 * @param response servlet response
 * @throws ServletException if a servlet-specific error occurs
 * @throws IOException if an I/O error occurs
 */
@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    processRequest(request, response);
}

/**
 * Handles the HTTP <code>POST</code> method.
 *
 * @param request servlet request
 * @param response servlet response
 * @throws ServletException if a servlet-specific error occurs
 * @throws IOException if an I/O error occurs
 */
@Override
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    processRequest(request, response);
}

/**
 * Returns a short description of the servlet.
 *
 * @return a String containing servlet description
 */
@Override
public String getServletInfo() {
    return "Short description";
}

// </editor-fold>
}

```

En principio, no contiene nada nuevo respecto al primer ejemplo, y casi el método `processRequest()` también es generado por NetBeans. Nótese que he añadido un método `isMail()`, que es un método de antes para validar una dirección de correo electrónico usando una expresión regular. El método `processRequest()` funciona en principio de la misma manera que en el primer ejemplo, y crea dinámicamente el HTML que se enviará como respuesta. En primer lugar, observe la declaración:

```
request.setCharacterEncoding("UTF-8");
```

Es necesario para descodificar correctamente las letras danesas. A continuación, debes fijarte en cómo captar los valores del formulario, por ejr ? :

```
String fname = request.getParameter("fname");
```

que determina el valor introducido como primer nombre. El resto está, en principio, fuera del camino y consiste en validar los campos enviados desde el formulario y, en caso de error, enviar un mensaje de error como respuesta. Así, la respuesta del servidor es dinámica porque depende de los datos enviados.

Está claro que el servlet podría hacer mucho más y podría, por ejemplo, guardar los datos enviados en una base de datos. Sólo sería cuestión de añadir algo de código Java.

Última modificación: lunes, 13 de marzo de 2023, 11:38